

Introduction to Information Retrieval

Image Encoding

Lecturer: Dr. LE Trung Nghia, Dr. NGUYEN Vinh Tiep

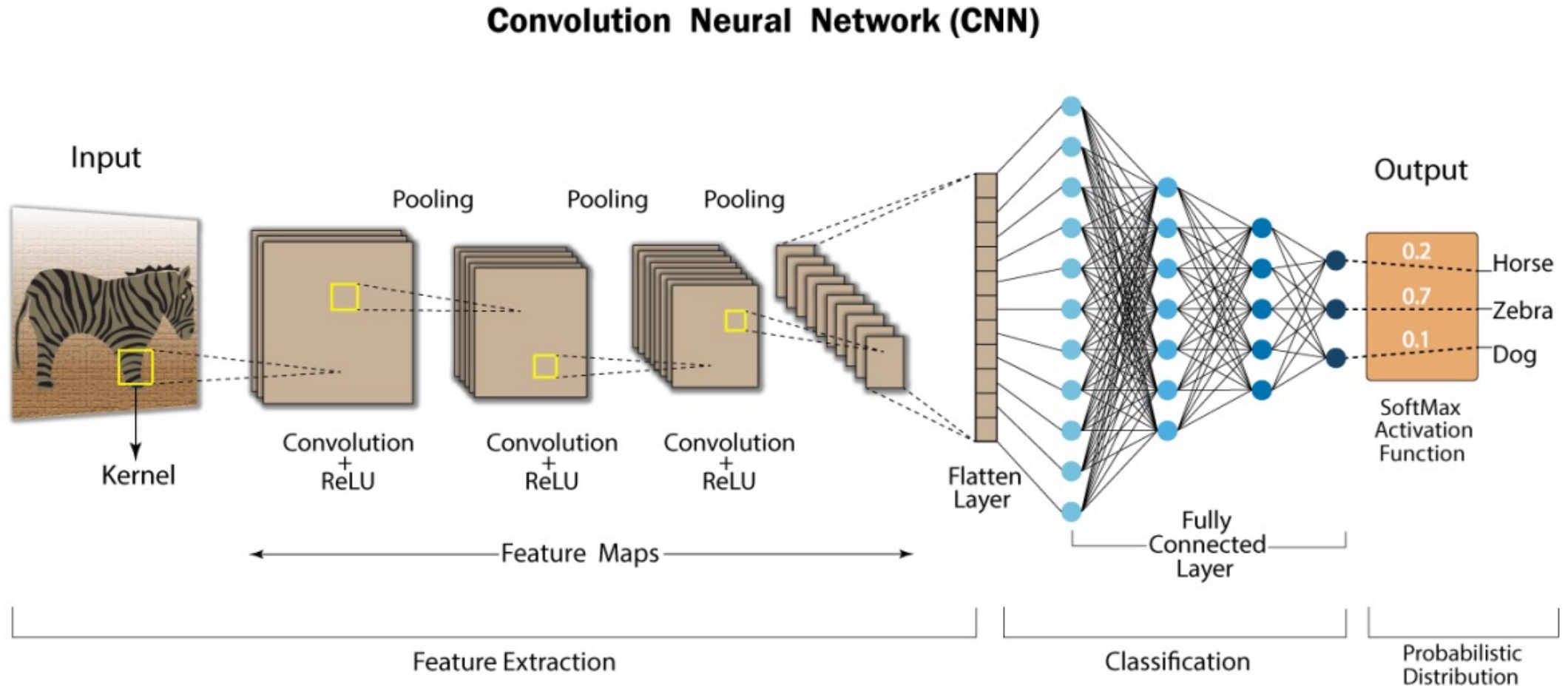
Disclaimer

- Disclaimer: This curriculum have been inspired and adapted from materials created by:
 - Richard Szeliski, “Computer Vision: Algorithms and Applications”, 2nd edition, 2022. <https://szeliski.org/Book/>

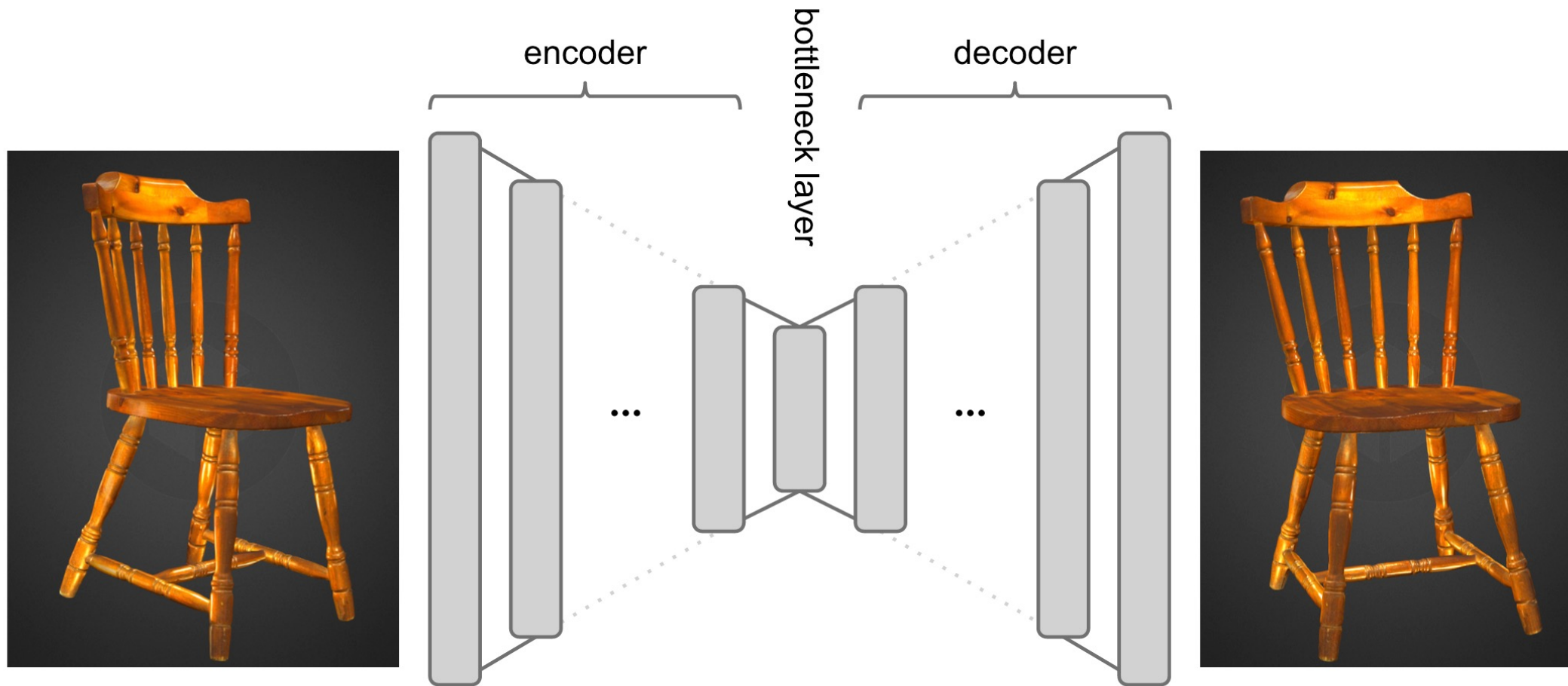
Feature Encoding

- Feature encoding: feature extraction
- Non deep learning image feature encoding:
 - Bag of visual words
 - Spatial pyramid representation (Lazebnik et al., 2006)
- Deep learning encoding:
 - Convolutional neural network
 - Autoencoder
 - Siamese network

Classification-base Feature Encoding

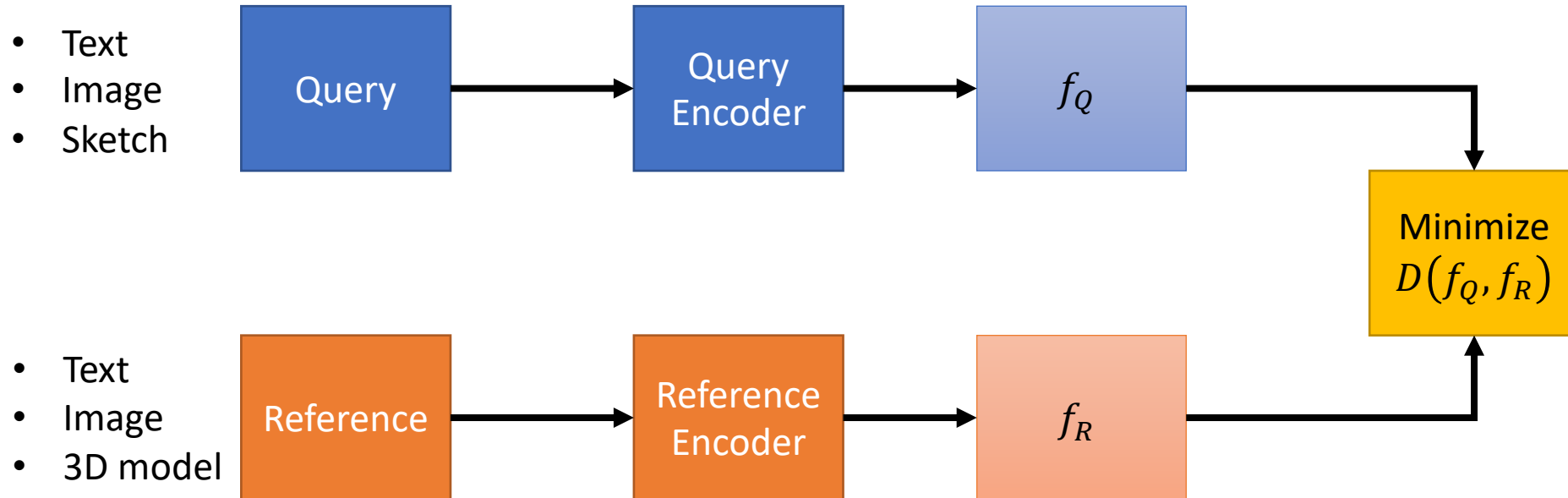


Autoencoder



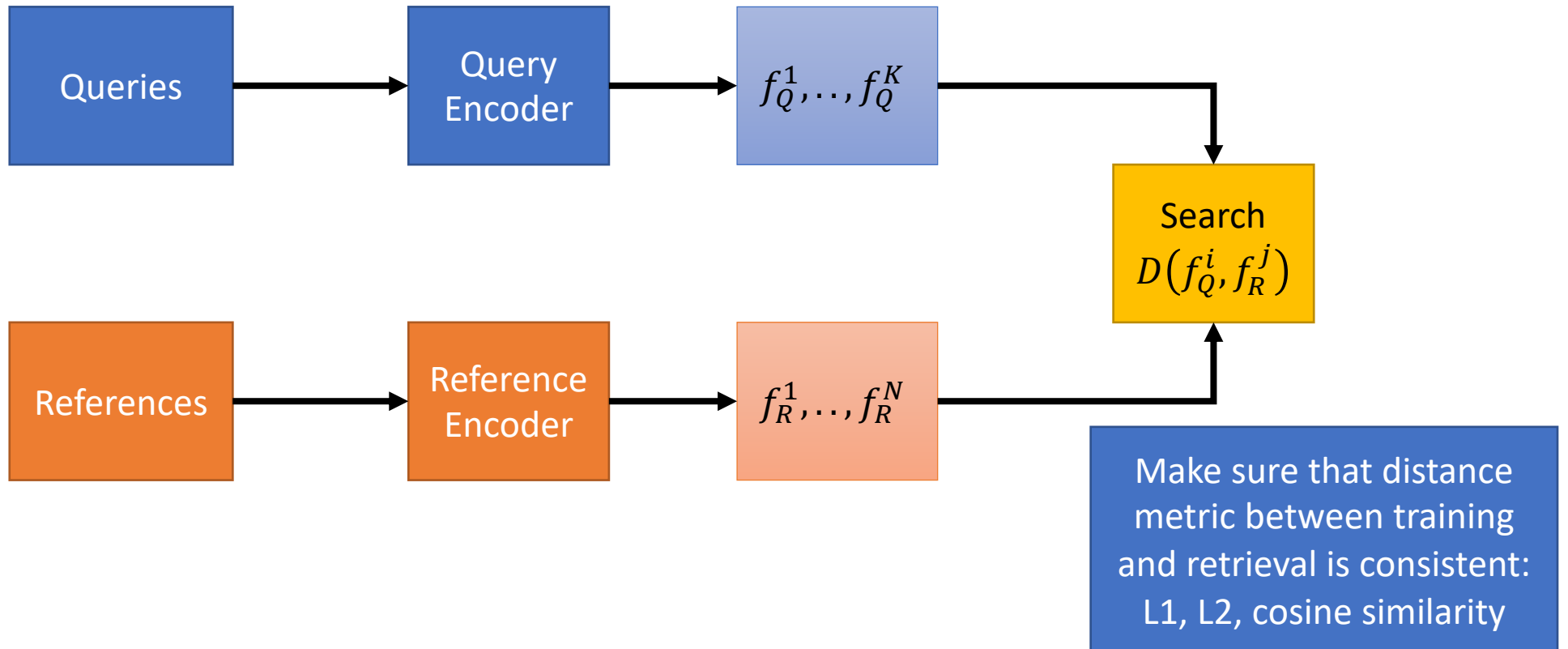
Content-Based Retrieval

- Given queries and references (database)
- Training:



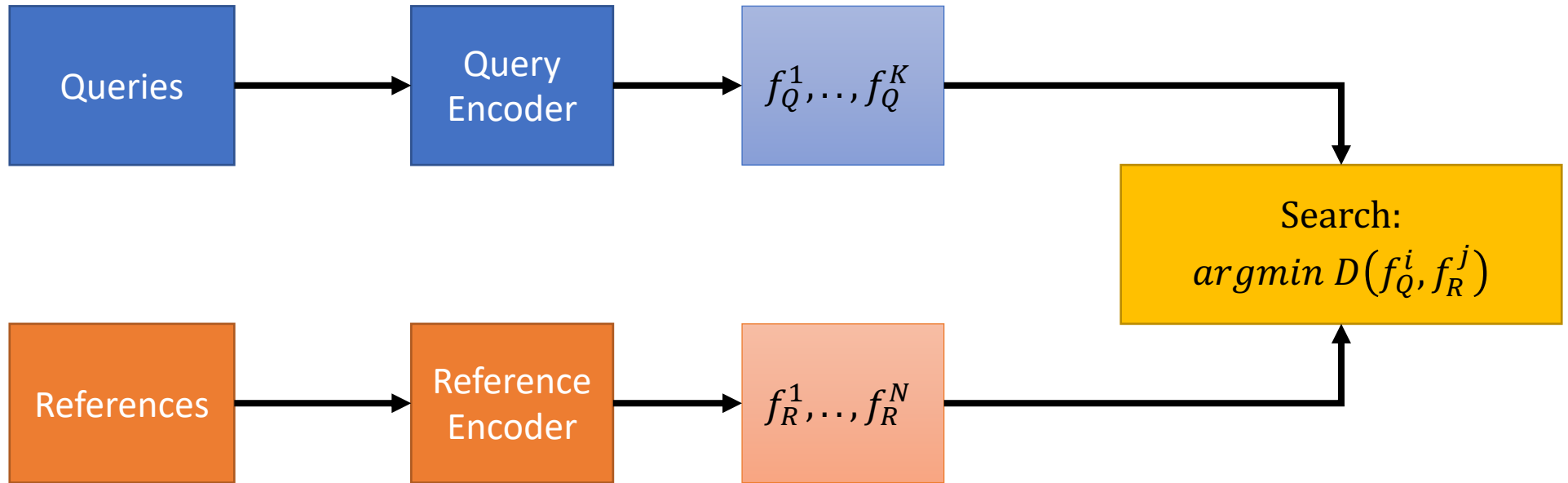
Content-Based Retrieval

- Given queries and references (database)
- Retrieval:



Content-Based Retrieval

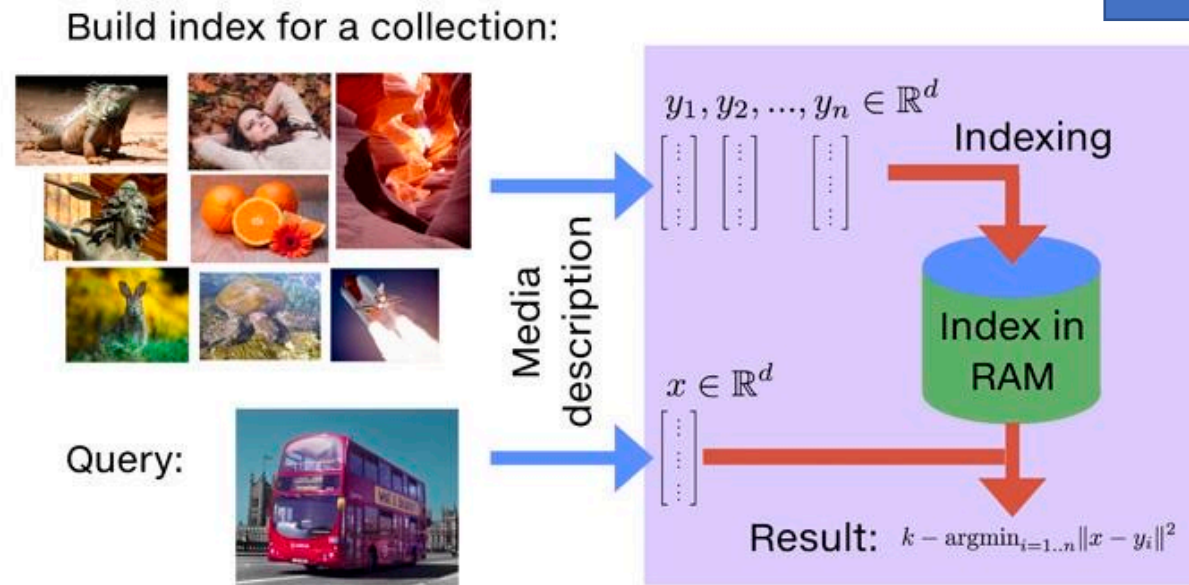
- How to search?



Facebook AI Similarity Search (FAISS)

- Return multiple similar results
- Freely optimize precision/accuracy and speed
- Batch processing

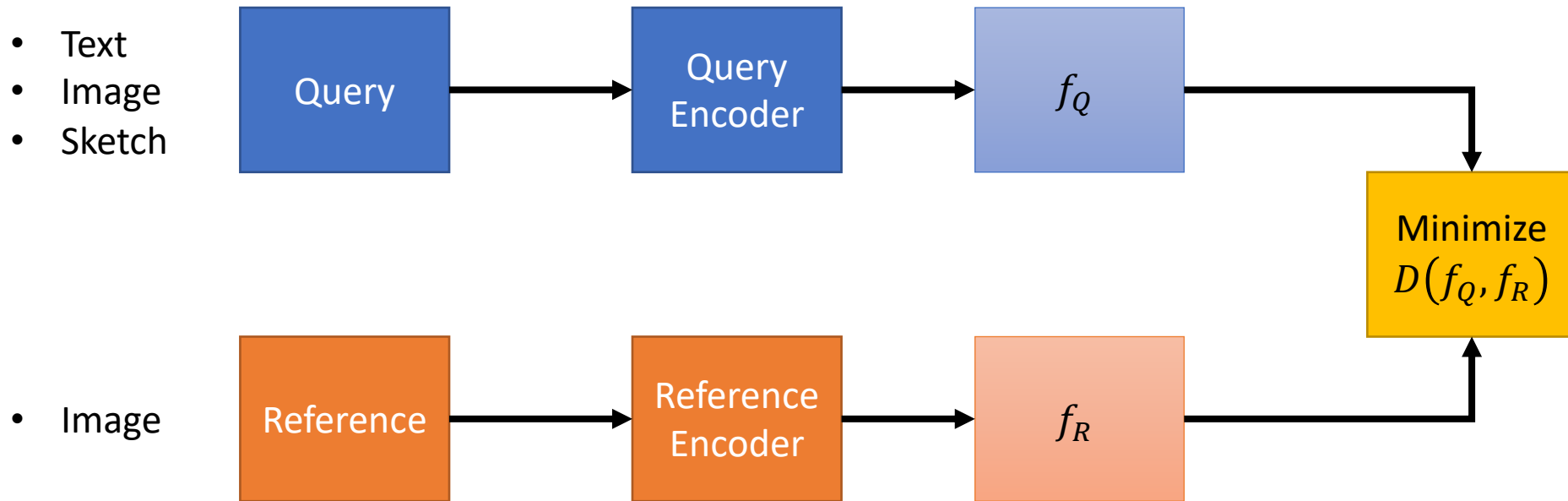
Remark: Normalize features before searching



Johnson Jeff et al., "Billion-scale similarity search with GPUs", Transactions on Big Data, 2019

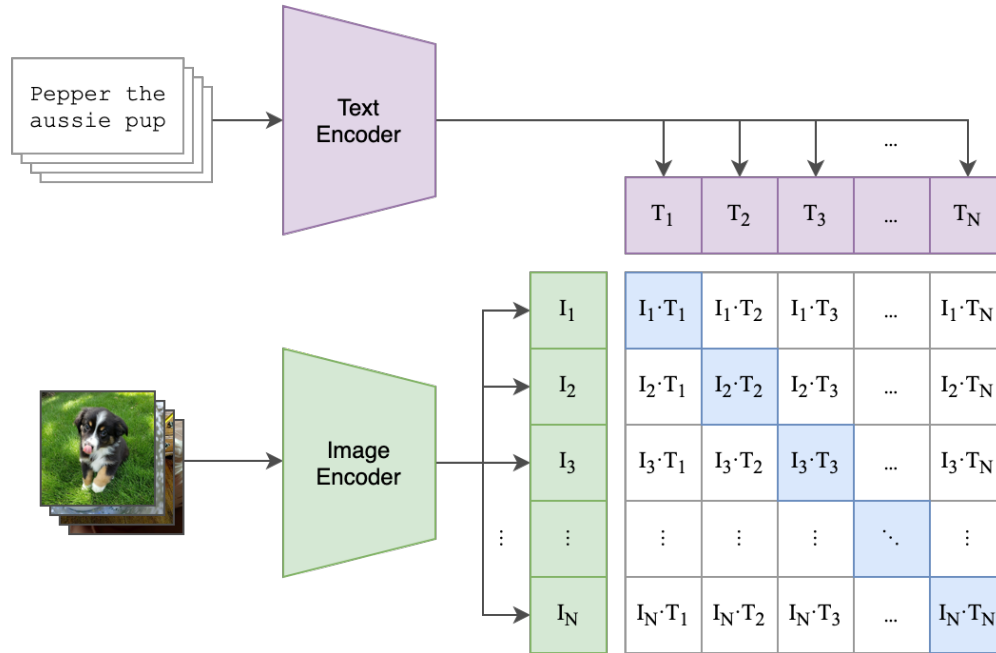
Content-Based Image Retrieval

- How to train encoder?
 - Data-driven
 - Task-driven

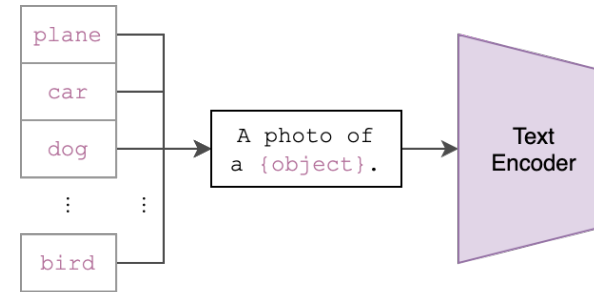


Contrastive Language-Image Pre-Training (CLIP)

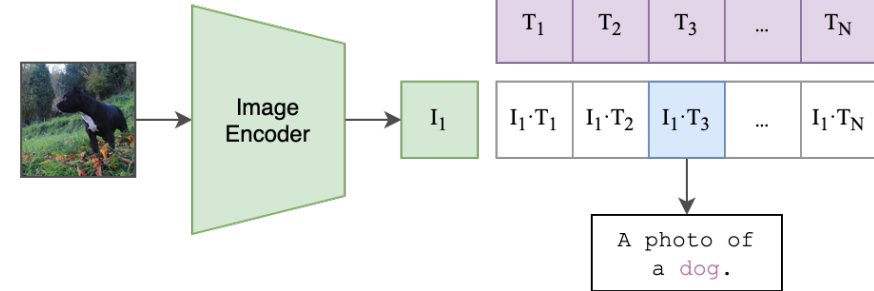
(1) Contrastive pre-training



(2) Create dataset classifier from label text



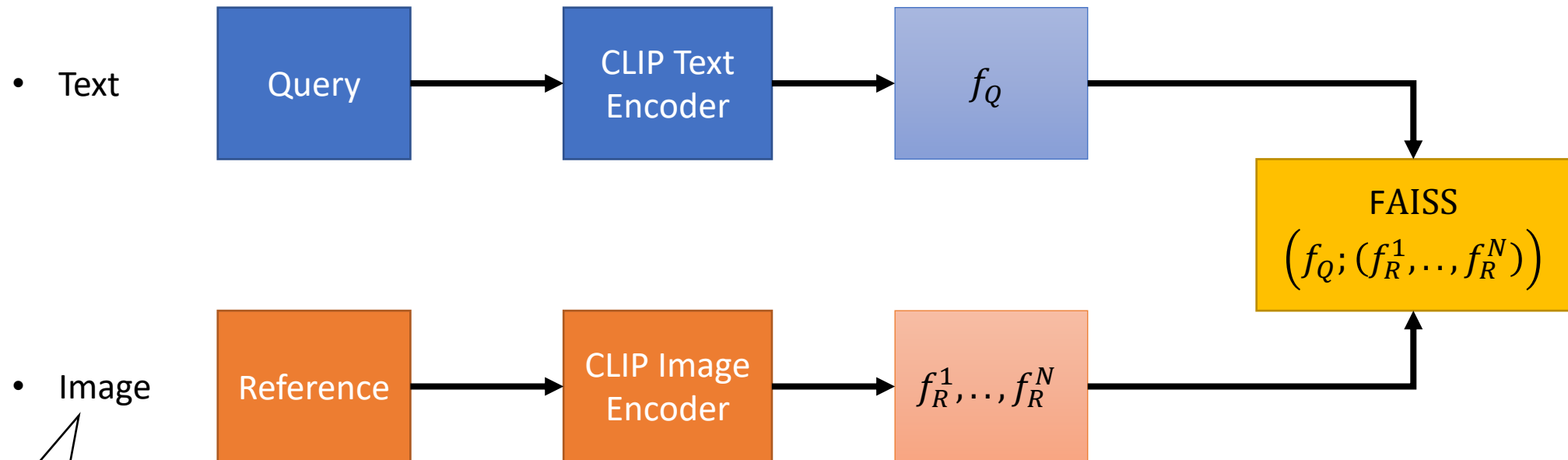
(3) Use for zero-shot prediction



- CLIP – OpenAI
- OpenCLIP - open source implementation of CLIP

Image Retrieval Using Pre-trained CLIP Features

- Retrieval:

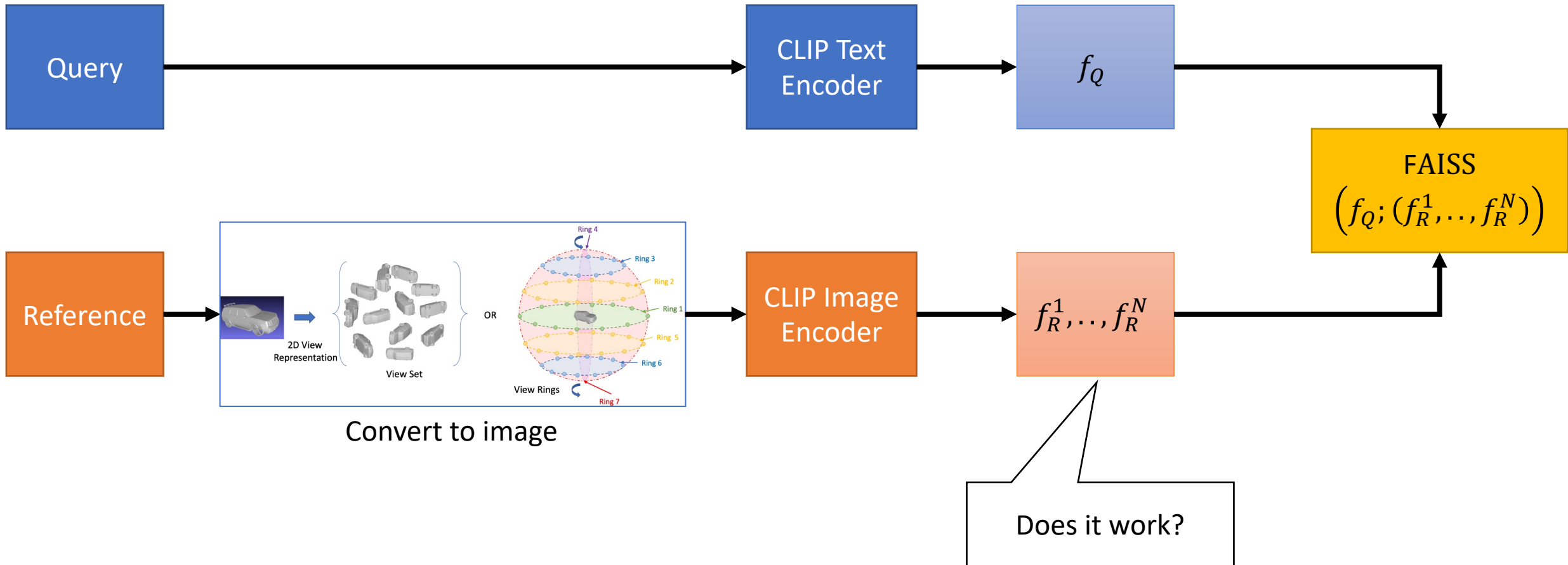


- Image

What if not image?

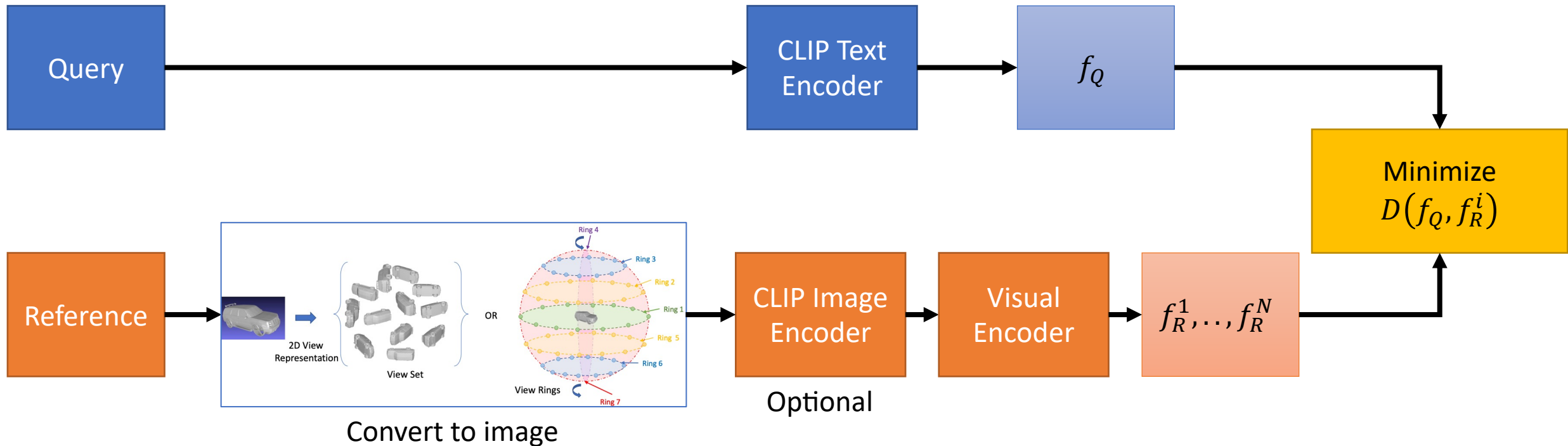
3D Object Retrieval Using CLIP Features

- Retrieval:



3D Object Retrieval Using CLIP Features

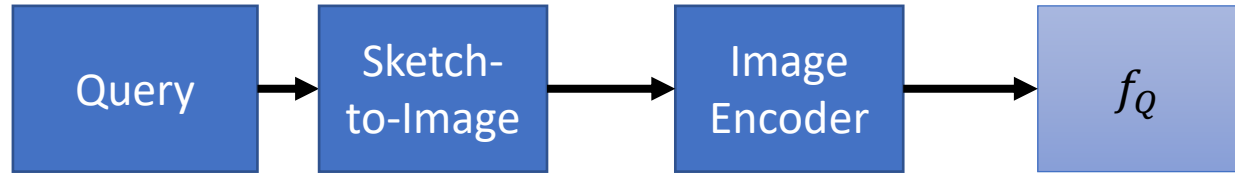
- Training:



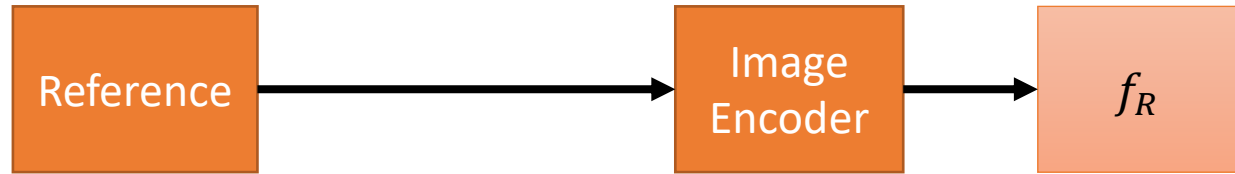
Sketch-based Image Retrieval

- Training:

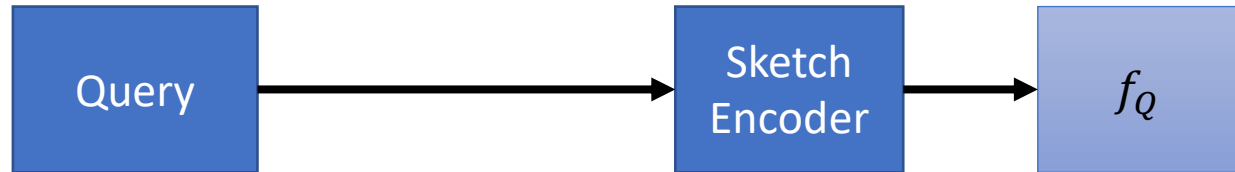
- Sketch



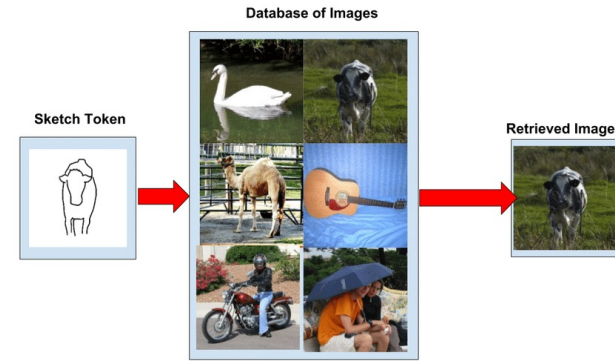
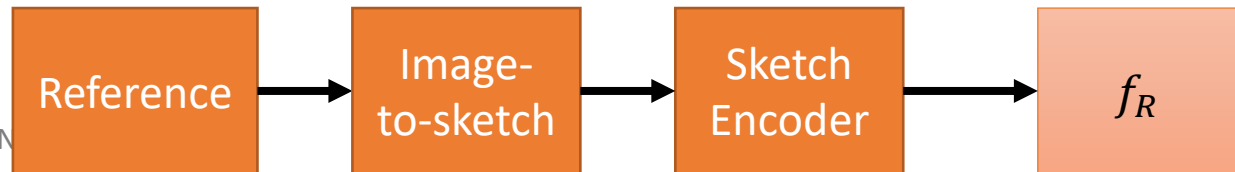
- Image



- Sketch

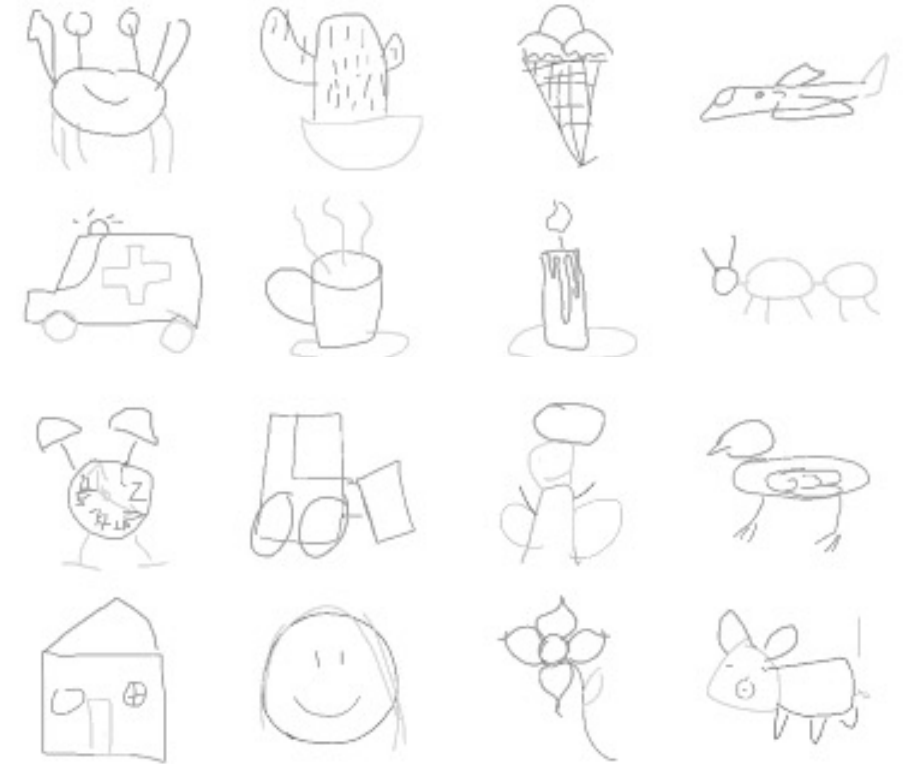
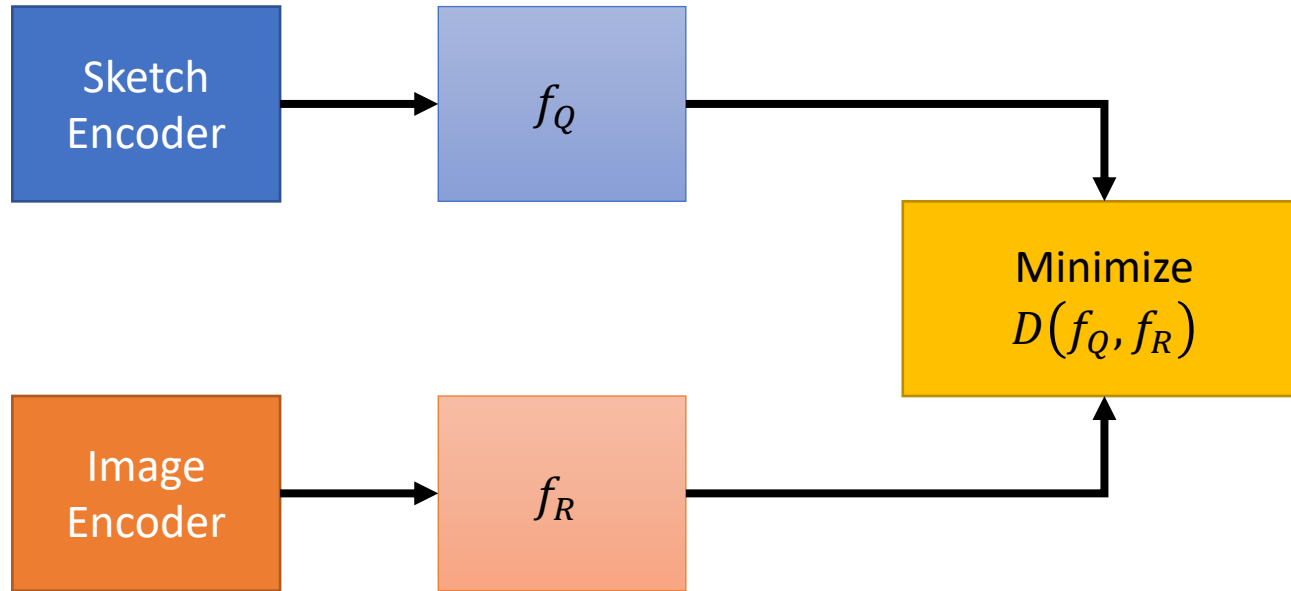


- Image



Sketch-based Image Retrieval

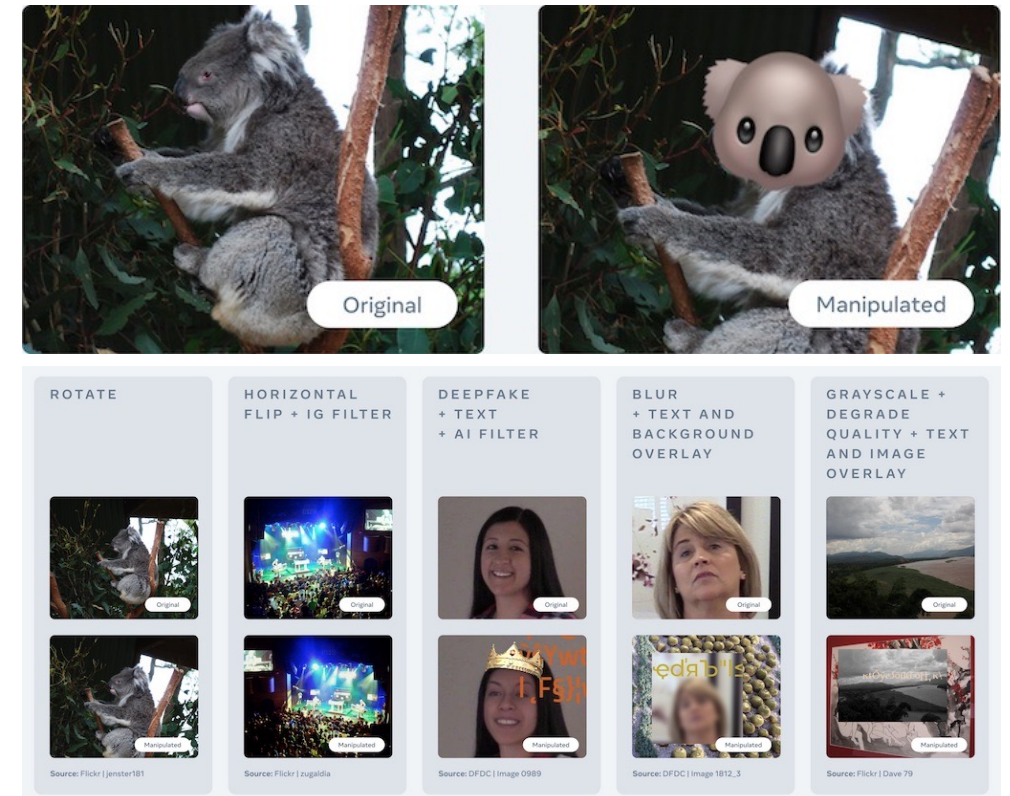
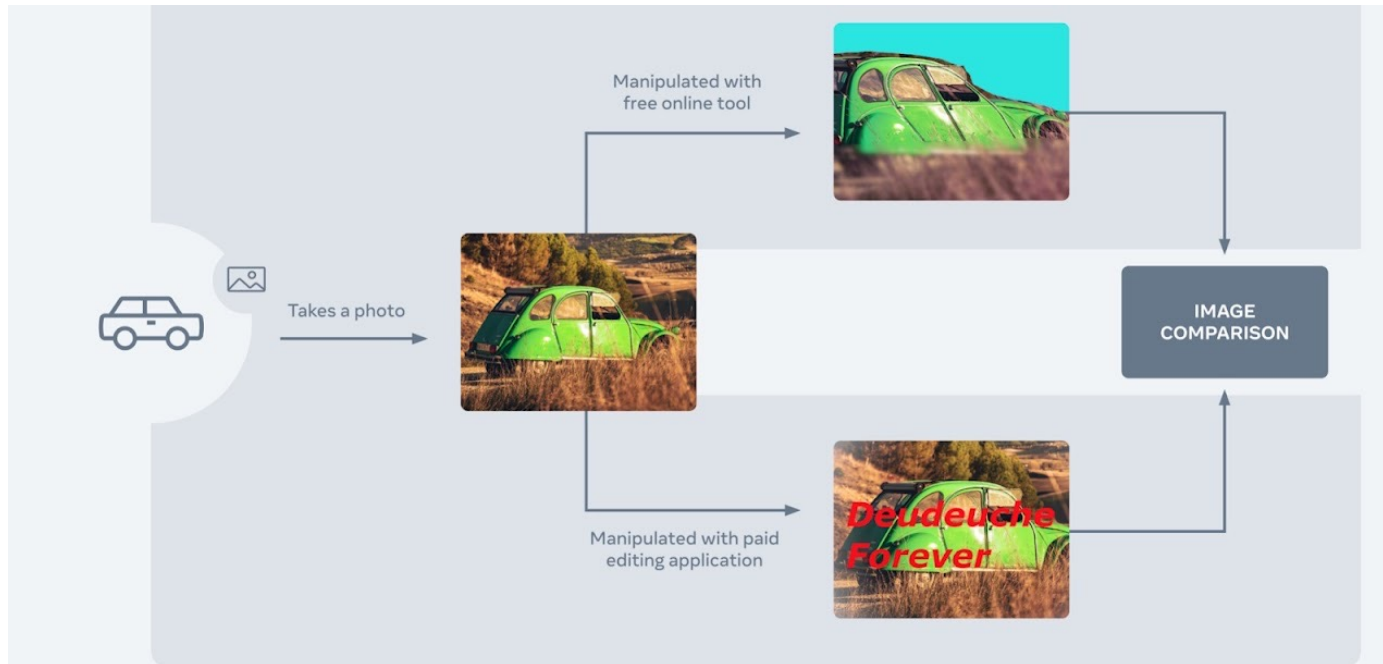
- Training:



=> Classification-based image retrieval

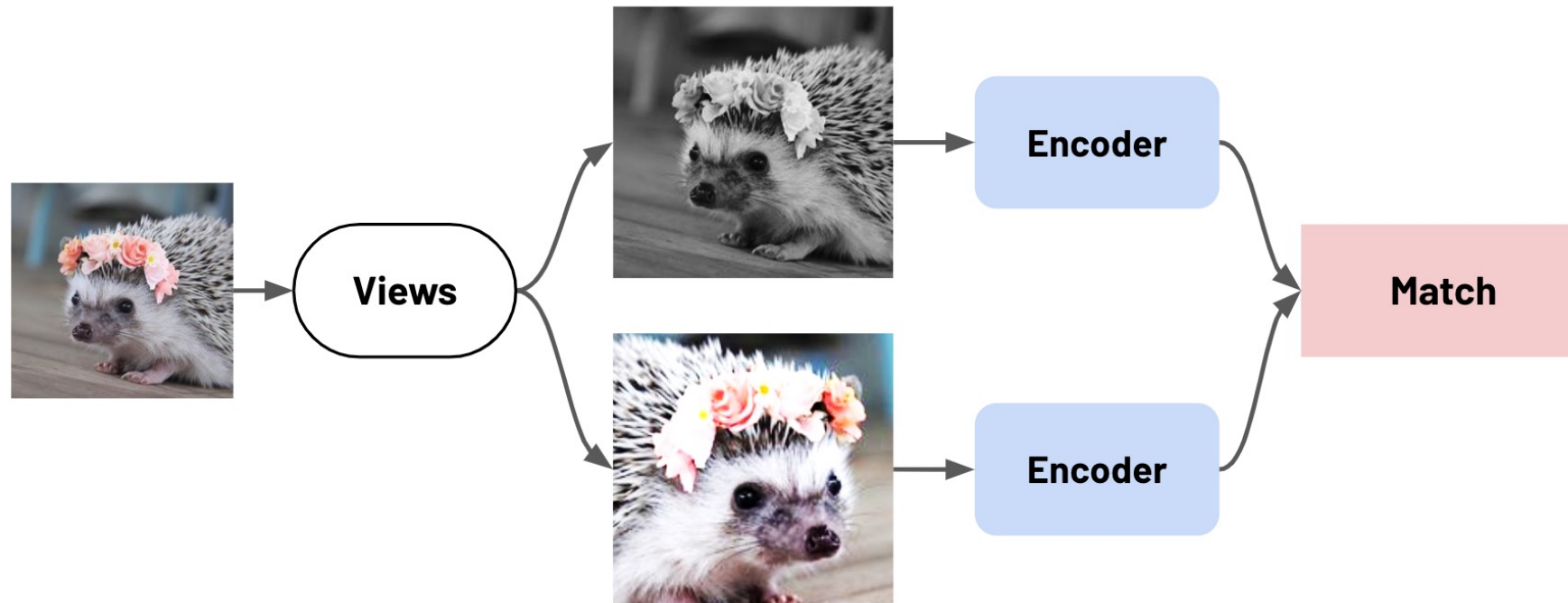
Image/Video Similarity Search (FAIR)

- Given manipulated image (query), find real one in large-scale database
- Data creation:



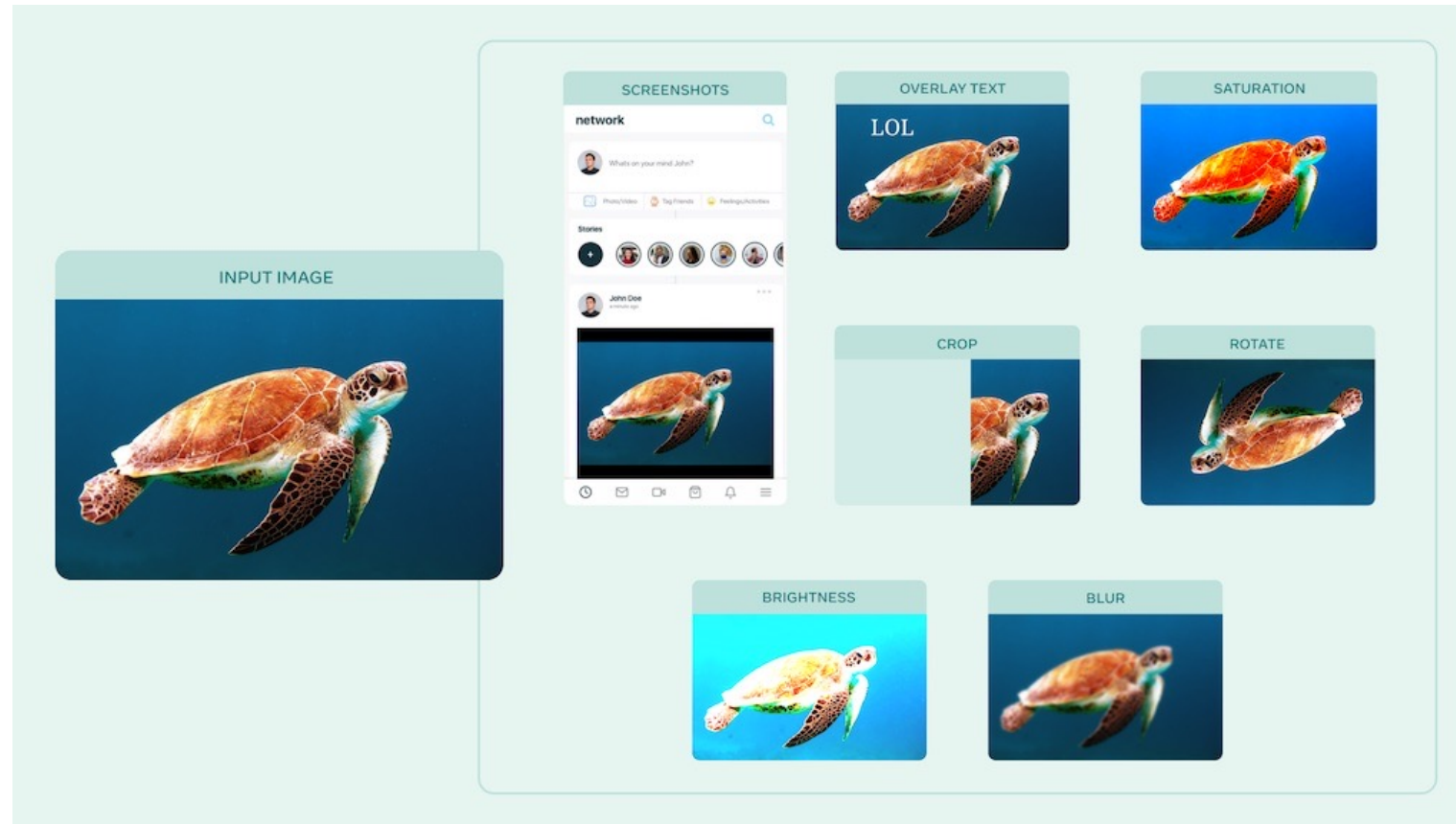
Image/Video Similarity Search (FAIR)

- Training:



AugLy

- <https://github.com/facebookresearch/AugLy/>

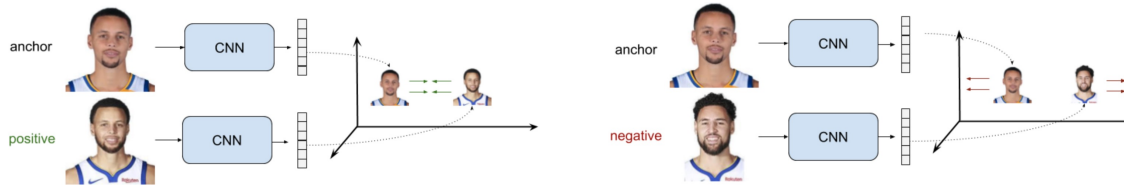


Siamese Network (Supervised Learning)

- ~~Binary cross entropy loss~~

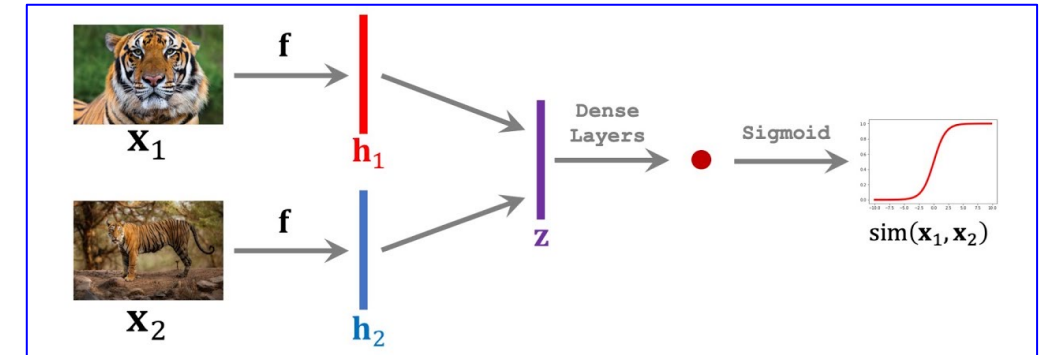
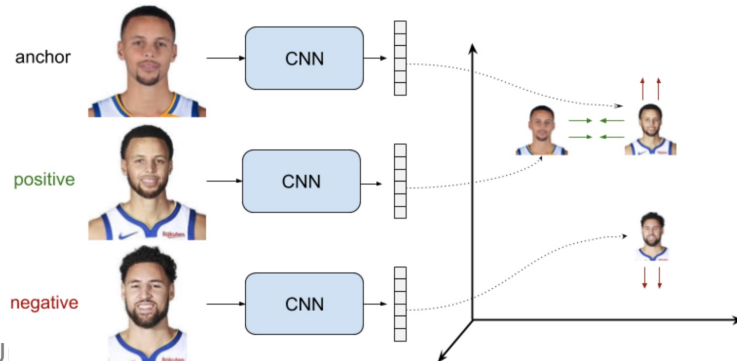
- Contrastive loss:

$$(1 - Y) \frac{1}{2} (D_w)^2 + (Y) \frac{1}{2} \max(0, m - D_w)^2$$

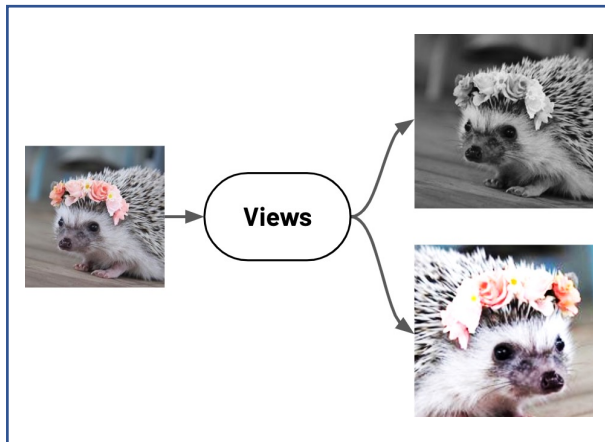
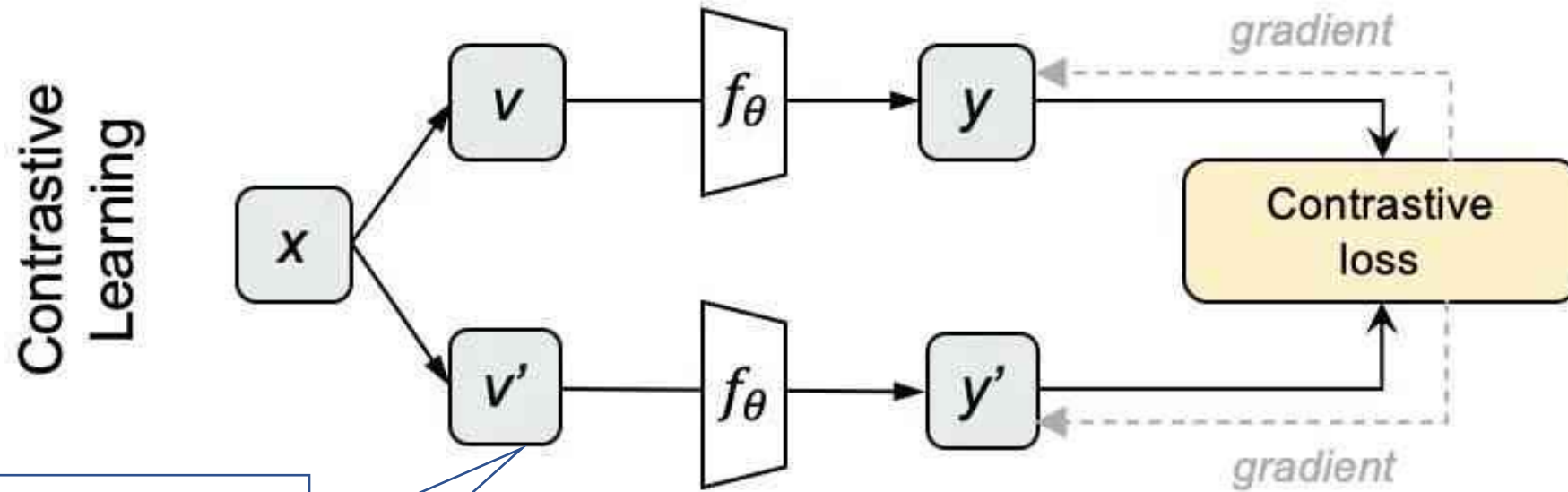


- Triplet loss:

$$L(A, P, N) = \max(\|f(A) - f(P)\|^2 - \|f(A) - f(N)\|^2 + \alpha, 0)$$



Siamese Network (Self-Supervised Learning)



SimCLR [ICML 20]

Algorithm 1 SimCLR's main learning algorithm.

input: batch size N , constant τ , structure of $f, g, \mathcal{T}$.
for sampled minibatch $\{\mathbf{x}_k\}_{k=1}^N$ **do**
 for all $k \in \{1, \dots, N\}$ **do**
 draw two augmentation functions $t \sim \mathcal{T}, t' \sim \mathcal{T}$
 # the first augmentation
 $\tilde{\mathbf{x}}_{2k-1} = t(\mathbf{x}_k)$
 $\mathbf{h}_{2k-1} = f(\tilde{\mathbf{x}}_{2k-1})$ # representation
 $\mathbf{z}_{2k-1} = g(\mathbf{h}_{2k-1})$ # projection
 # the second augmentation
 $\tilde{\mathbf{x}}_{2k} = t'(\mathbf{x}_k)$
 $\mathbf{h}_{2k} = f(\tilde{\mathbf{x}}_{2k})$ # representation
 $\mathbf{z}_{2k} = g(\mathbf{h}_{2k})$ # projection
 end for
 for all $i \in \{1, \dots, 2N\}$ and $j \in \{1, \dots, 2N\}$ **do**
 $s_{i,j} = \mathbf{z}_i^\top \mathbf{z}_j / (\|\mathbf{z}_i\| \|\mathbf{z}_j\|)$ # pairwise similarity
 end for
 define $\ell(i, j)$ **as** $\ell(i, j) = -\log \frac{\exp(s_{i,j}/\tau)}{\sum_{k=1}^{2N} \mathbb{1}_{[k \neq i]} \exp(s_{i,k}/\tau)}$
 $\mathcal{L} = \frac{1}{2N} \sum_{k=1}^N [\ell(2k-1, 2k) + \ell(2k, 2k-1)]$
 update networks f and g to minimize $\mathcal{L}$
end for
return encoder network $f(\cdot)$, and throw away $g(\cdot)$

A Simple Framework for Contrastive Learning of Visual Representations

<https://github.com/google-research/simclr>

Barlow Twins [ICML 2021]

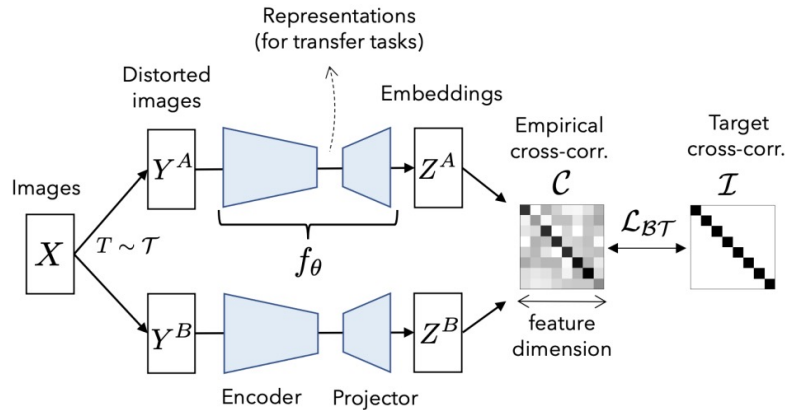


Figure 1. BARLOW TWINS’s objective function measures the cross-correlation matrix between the embeddings of two identical networks fed with distorted versions of a batch of samples, and tries to make this matrix close to the identity. This causes the embedding vectors of distorted versions of a sample to be similar, while minimizing the redundancy between the components of these vectors. BARLOW TWINS is competitive with state-of-the-art methods for self-supervised learning while being conceptually simpler, naturally avoiding trivial constant (i.e. collapsed) embeddings, and being robust to the training batch size.

Algorithm 1 PyTorch-style pseudocode for Barlow Twins.

```
# f: encoder network
# lambda: weight on the off-diagonal terms
# N: batch size
# D: dimensionality of the embeddings
#
# mm: matrix-matrix multiplication
# off_diagonal: off-diagonal elements of a matrix
# eye: identity matrix

for x in loader: # load a batch with N samples
    # two randomly augmented versions of x
    y_a, y_b = augment(x)

    # compute embeddings
    z_a = f(y_a) # Nx D
    z_b = f(y_b) # Nx D

    # normalize repr. along the batch dimension
    z_a_norm = (z_a - z_a.mean(0)) / z_a.std(0) # Nx D
    z_b_norm = (z_b - z_b.mean(0)) / z_b.std(0) # Nx D

    # cross-correlation matrix
    c = mm(z_a_norm.T, z_b_norm) / N # DxD

    # loss
    c_diff = (c - eye(D)).pow(2) # DxD
    # multiply off-diagonal elems of c_diff by lambda
    off_diagonal(c_diff).mul_(lambda)
    loss = c_diff.sum()

    # optimization step
    loss.backward()
    optimizer.step()
```

Barlow Twins: Self-Supervised Learning via Redundancy Reduction

<https://github.com/facebookresearch/barlowtwins>

VICReg [ICLR 2022]

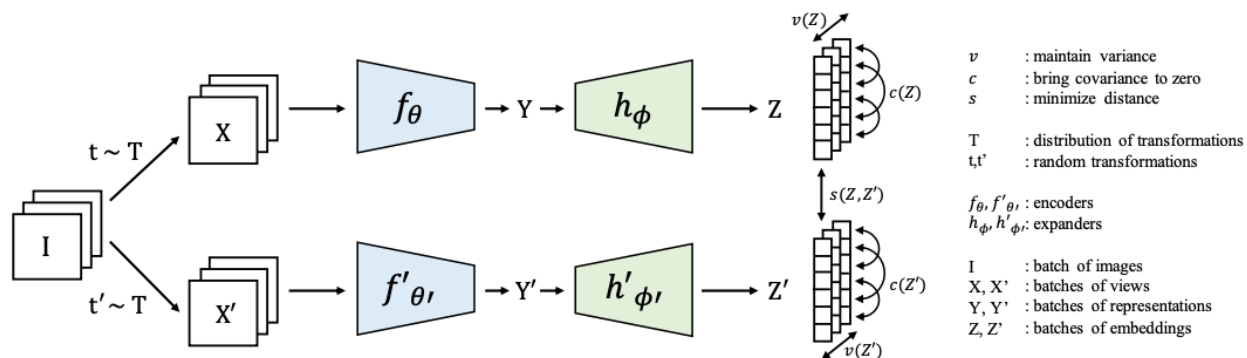


Figure 1: VICReg: joint embedding architecture with variance, invariance and covariance regularization. Given a batch of images I , two batches of different views X and X' are produced and are then encoded into representations Y and Y' . The representations are fed to an expander producing the embeddings Z and Z' . The distance between two embeddings from the same image is minimized, the variance of each embedding variable over a batch is maintained above a threshold, and the covariance between pairs of embedding variables over a batch are attracted to zero, decorrelating the variables from each other. Although the two branches do not require identical architectures nor share weights, in most of our experiments, they are Siamese with shared weights: the encoders are ResNet-50 backbones with output dimension 2048. The expanders have 3 fully-connected layers of size 8192.

Algorithm 1: VICReg pytorch pseudocode.

```

# f: encoder network, lambda, mu, nu: coefficients of the
# invariance, variance and covariance losses, N: batch size
# D: dimension of the representations
# mse_loss: Mean square error loss function, off_diagonal:
# off-diagonal elements of a matrix, relu: ReLU activation
# function

for x in loader: # load a batch with N samples
    # two randomly augmented versions of x
    x_a, x_b = augment(x)

    # compute representations
    z_a = f(x_a) # N x D
    z_b = f(x_b) # N x D

    # invariance loss
    sim_loss = mse_loss(z_a, z_b)

    # variance loss
    std_z_a = torch.sqrt(z_a.var(dim=0) + 1e-04)
    std_z_b = torch.sqrt(z_b.var(dim=0) + 1e-04)
    std_loss = torch.mean(relu(1 - std_z_a)) + torch.mean(
        relu(1 - std_z_b))

    # covariance loss
    z_a = z_a - z_a.mean(dim=0)
    z_b = z_b - z_b.mean(dim=0)
    cov_z_a = (z_a.T @ z_a) / (N - 1)
    cov_z_b = (z_b.T @ z_b) / (N - 1)
    cov_loss = off_diagonal(cov_z_a).pow(2).sum() / D
    + off_diagonal(cov_z_b).pow(2).sum() / D

    # loss
    loss = lambda * sim_loss + mu * std_loss + nu * cov_loss

    # optimization step
    loss.backward()
    optimizer.step()
    
```

VICReg: Variance-Invariance-Covariance Regularization for Self-Supervised Learning

<https://github.com/facebookresearch/vicreg>

BYOL [NeurIPS 2020]

Polyak Averaging is an optimization technique that sets final parameters to an average of (recent) parameters visited in the optimization trajectory. Specifically if in t iterations we have parameters $\theta_1, \theta_2, \dots, \theta_t$, then Polyak Averaging suggests setting

$$\theta_t = \frac{1}{t} \sum_i \theta_i$$

Polyak Averaging: Motivation

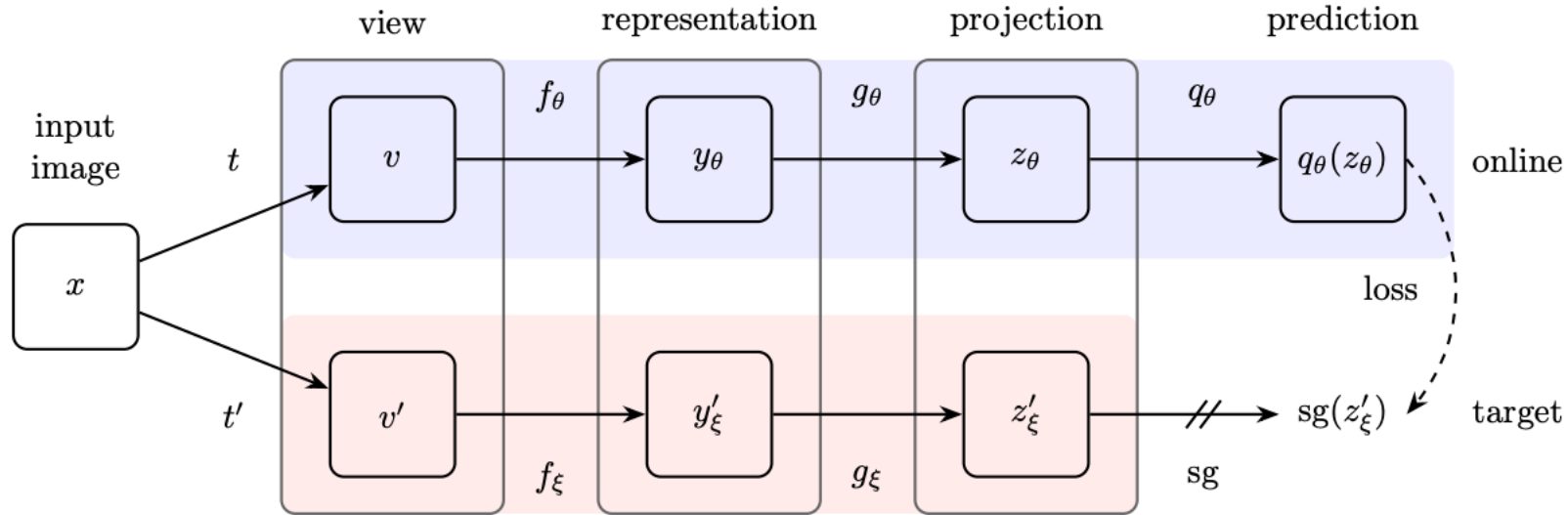
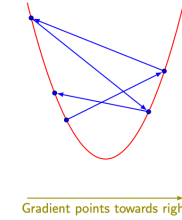
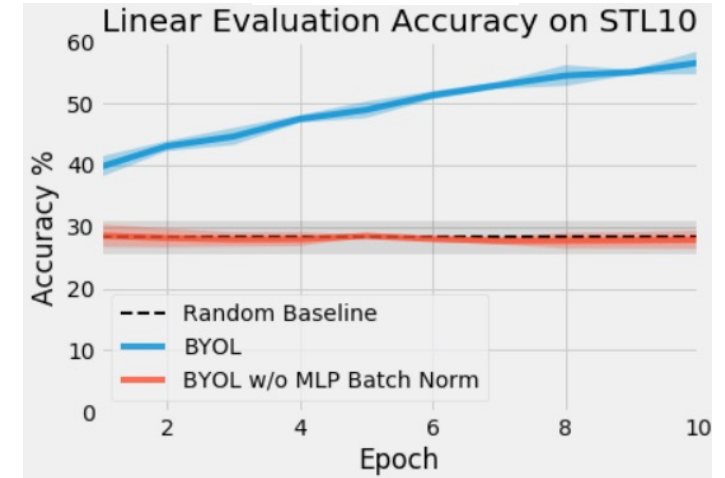


Figure 2: BYOL’s architecture. BYOL minimizes a similarity loss between $q_\theta(z_\theta)$ and $sg(z'_\xi)$, where θ are the trained weights, ξ are an exponential moving average of θ and sg means stop-gradient. At the end of training, everything but f_θ is discarded, and y_θ is used as the image representation.

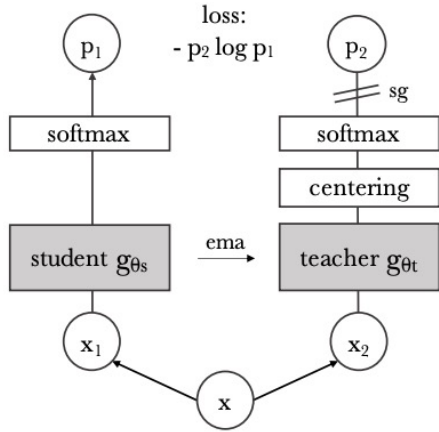
- ξ is updated by polyak averaging: $\xi_i \leftarrow \tau_i \xi_{i-1} + (1 - \tau_i) \theta_i$
- $\tau_i \leftarrow 1 - \frac{(1-\tau_0) * (\cos \frac{i\pi}{n} + 1)}{2}$ with $\tau_0(n = 200) = 0.97$ and $\tau_0(n = 1000) = 0.996$

Bootstrap Your Own Latent

<https://github.com/lucidrains/byol-pytorch>



DINO [ICCV 2021]



Multi-crop training: input to teacher network is complete image while student network takes in complete image as well as random crops of whole image.

Algorithm 1 DINO PyTorch pseudocode w/o multi-crop.

```
# gs, gt: student and teacher networks
# C: center (K)
# tps, tpt: student and teacher temperatures
# l, m: network and center momentum rates
gt.params = gs.params
for x in loader: # load a minibatch x with n samples
    x1, x2 = augment(x), augment(x) # random views

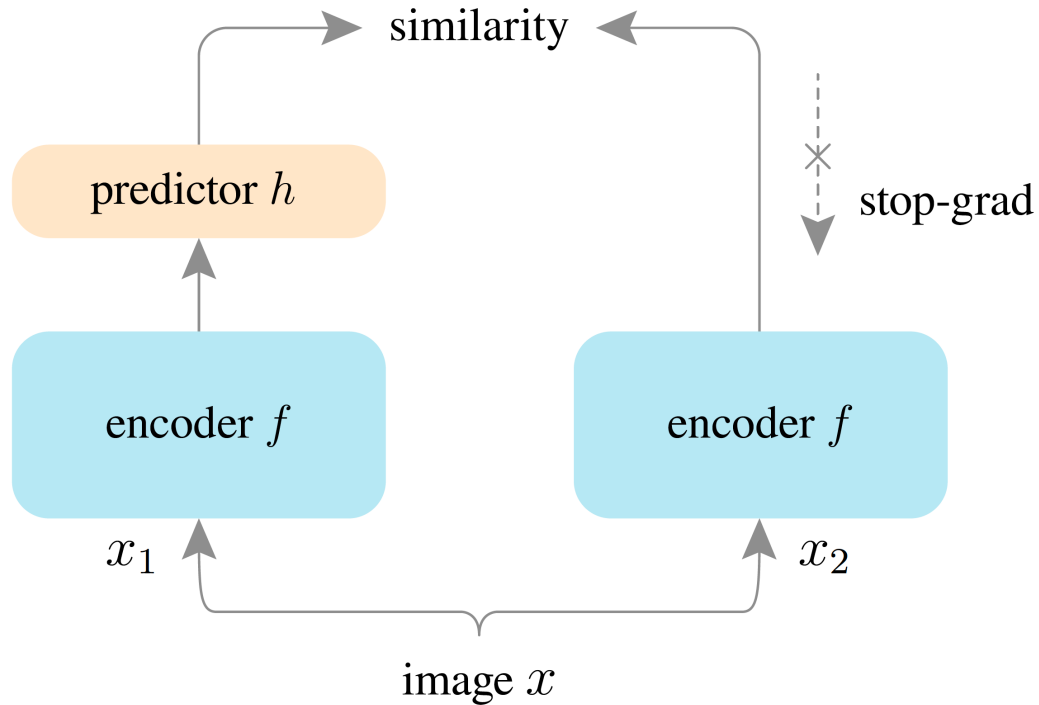
    s1, s2 = gs(x1), gs(x2) # student output n-by-K
    t1, t2 = gt(x1), gt(x2) # teacher output n-by-K

    loss = H(t1, s2)/2 + H(t2, s1)/2
    loss.backward() # back-propagate

    # student, teacher and center updates
    update(gs) # SGD
    gt.params = l*gt.params + (1-l)*gs.params
    C = m*C + (1-m)*cat([t1, t2]).mean(dim=0)

def H(t, s):
    t = t.detach() # stop gradient
    s = softmax(s / tps, dim=1)
    t = softmax((t - C) / tpt, dim=1) # center + sharpen
    return - (t * log(s)).sum(dim=1).mean()
```

SimSiam [CVPR 2021]



Algorithm 1 SimSiam Pseudocode, PyTorch-like

```
# f: backbone + projection mlp
# h: prediction mlp

for x in loader: # load a minibatch x with n samples
    x1, x2 = aug(x), aug(x) # random augmentation
    z1, z2 = f(x1), f(x2) # projections, n-by-d
    p1, p2 = h(z1), h(z2) # predictions, n-by-d

    L = D(p1, z2)/2 + D(p2, z1)/2 # loss

    L.backward() # back-propagate
    update(f, h) # SGD update

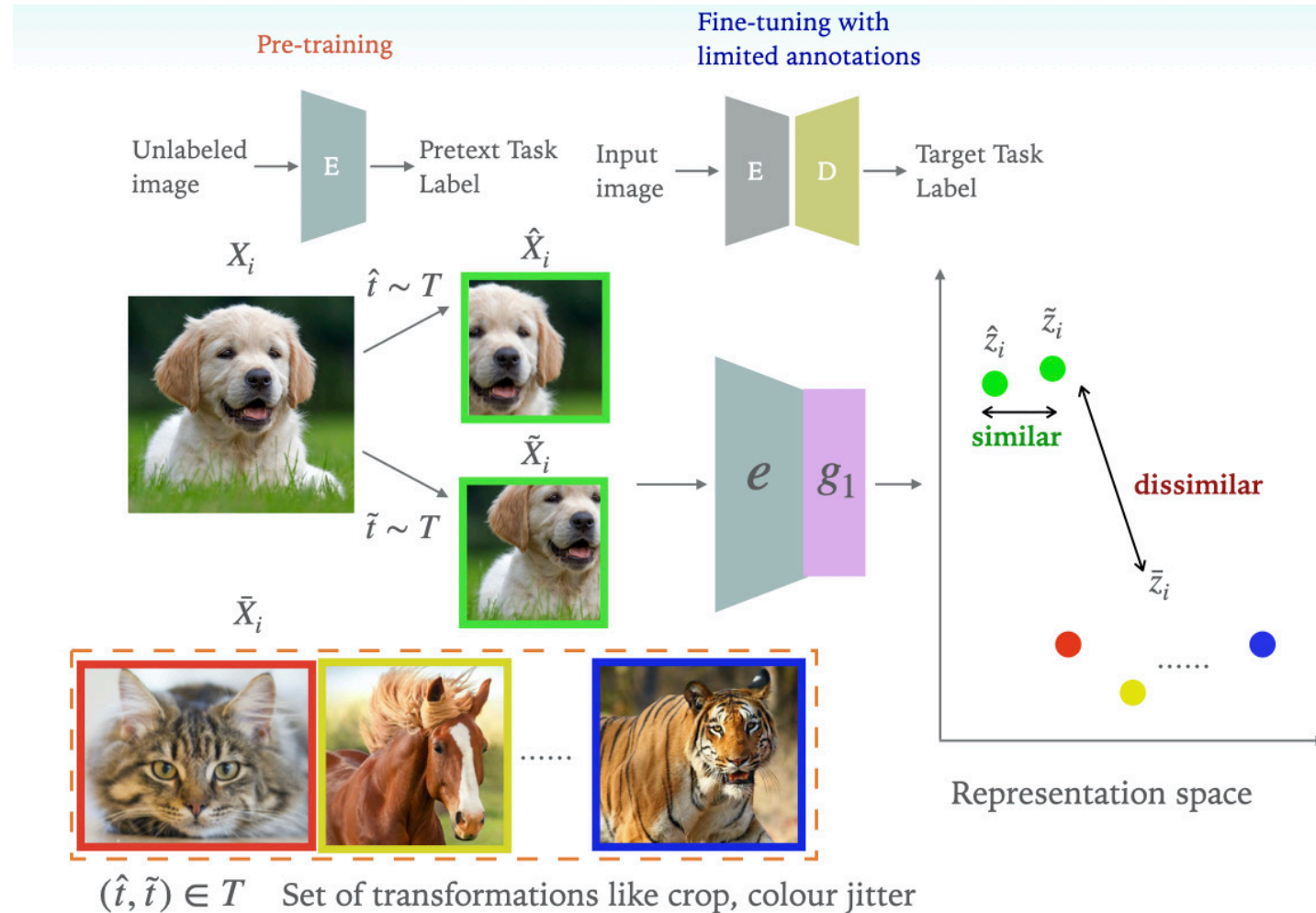
def D(p, z): # negative cosine similarity
    z = z.detach() # stop gradient

    p = normalize(p, dim=1) # l2-normalize
    z = normalize(z, dim=1) # l2-normalize
    return -(p*z).sum(dim=1).mean()
```

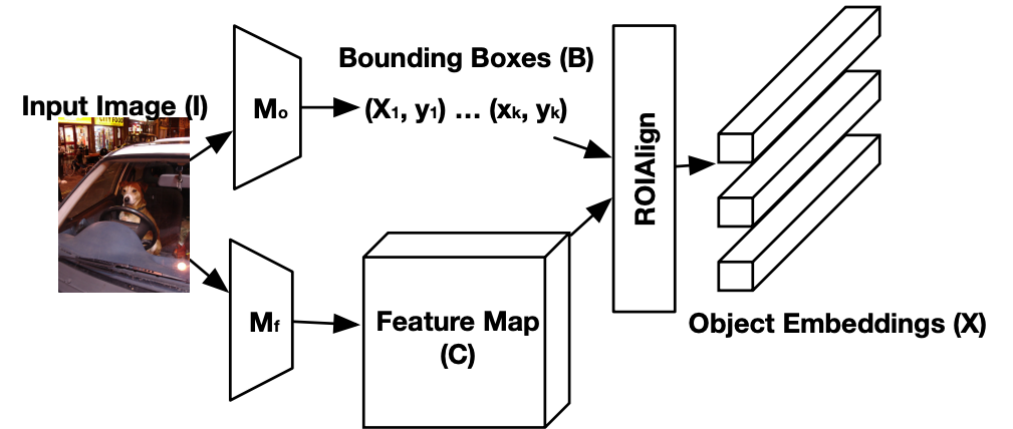
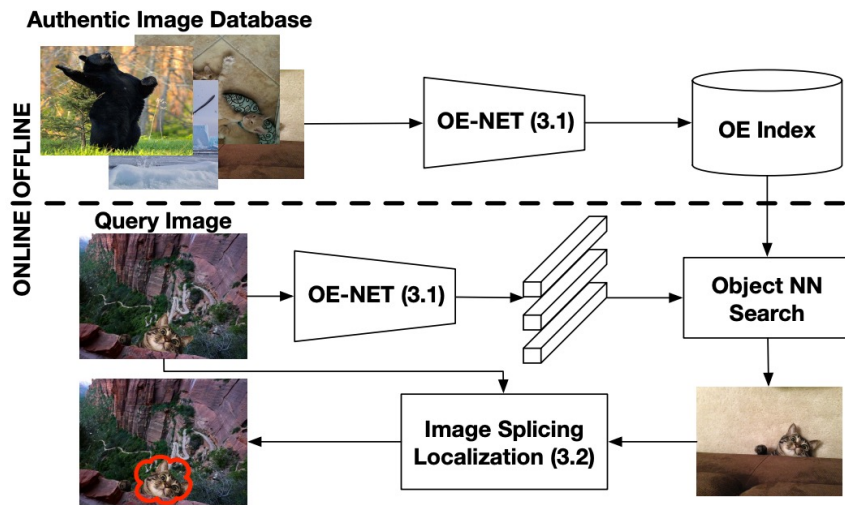
X. Chen et al., “Exploring Simple Siamese Representation Learning”, CVPR 2021

<https://github.com/facebookresearch/simsiam>

How to Use Contrastive Learning



Use Object-Level Features for Retrieval



Given a query image represented by n object embeddings $I_q = \{x_1^q, x_2^q, \dots, x_n^q\}$, and a database image $I_d = \{x_1^d, x_2^d, \dots, x_m^d\}$, the distance between the image pair is calculated as the minimum distance between pairwise object embeddings:

$$D(I_q, I_d) = \min_{i,j} \|x_i^q - x_j^d\|_2^2. \quad (7)$$